

# The Stack and Introduction to Procedures

Course Code: COE 3205

Course Title: Computer Organization & Architecture



**Dept. of Computer Science**  
**Faculty of Science and Technology**

|                  |          |                 |           |                  |                   |
|------------------|----------|-----------------|-----------|------------------|-------------------|
| <b>Lab No:</b>   | <b>9</b> | <b>Week No:</b> | <b>10</b> | <b>Semester:</b> | <b>Fall 22-23</b> |
| <b>Lecturer:</b> |          |                 |           |                  |                   |

# Lab Outline



1. **Overview of Stack**
2. **PUSH AND PUSHF Instructions**
3. **POP AND POPF**

# Stack

## Overview



- The stack segment of a program is used for **temporary storage** of data and addresses.
- The **PUSH** and **POP** Instructions are used to add and remove words from the stack respectively.
- A stack can be used to reverse a list of data because the last word to be added to the stack Is the first to be removed(**LIFO**),

# Stack

## Overview



- A stack is **one-dimensional** data structure.
- Items are added and removed from **one end** of the structure; that is, it is processed in a "**last-in, first-out**" manner.
- The **most recent addition** to the stack is called the **top** of the stack.
- A familiar example is a Stack of dishes; the last dish to go on the stack is the top one, and it's the only one that can be removed easily.

# Stack

## Overview



- A program must **set aside a block of memory** to hold the stack.
- We have been doing this by declaring a stack segment. For example, `.STACK 100H`
- When the program is assembled and loaded in memory, SS will contain the segment number of the stack segment.
- For the preceding Stack declaration, SP, the stack pointer, is initialized to 100h.
- This represents empty stack position.

# Stack

## Overview



- When the stack is empty, SP contains the offset address of the top of the stack.

# PUSH AND PUSHF Conti..



- PUSH is used to add new word to the stack.
- The syntax is:  
PUSH Source (i.e. PUSH AX)
- SP is decreased by 2
- A copy of source content is moved to the address specified by SS:SP.

# PUSH AND PUSHF



- Initially, SP contains the offset address of memory location.
- The first PUSH decreases SP by 2 and point to the LAST WORD in the STACK segment.
- PUSHF has no operands and pushes the contents of the flag register to the stack.

# POP AND POPF



- POP is used to remove an item from the stack.
- The syntax is:  
POP destination (i.e. POP AX)
- The content of SS:SP (the top of the stack) is moved to the destination.
- SP is Increased by 2.

# POP AND POPF



- The Instruction POPF, pops the top of the stack into the FLAGS register.
- There is no effect of PUSH, PUSHF. POP, POPF on the flags.
- Note that PUSH and POP are word operations, so a byte Instruction(i.e. PUSH DL or PUSH 2) is illegal.
- For INT 21h DOS saves instructions in STACK before execution.

# Lab Tasks

## Task: 1



- Suppose the register  $ax = 5$ ,  $bx = 6$ , Swap the numbers of  $ax$  and  $bx$  so that  $ax$  gets 6 and  $bx$  gets 5. use the concept of Stack. Push and Pop instructions must use.

- Before Swap:

| registers |  | H  | L  |
|-----------|--|----|----|
| AX        |  | 00 | 05 |
| BX        |  | 00 | 06 |

- After Swap :

| registers |  | H  | L  |
|-----------|--|----|----|
| AX        |  | 00 | 06 |
| BX        |  | 00 | 05 |

# Lab Tasks

Task: 2



- Suppose that **AX= 1234h**, **BX= 5678h**, **CX = 9ABCh**, and **SP= 100h**. Write an assembly program to find out the contents of **AX**, **BX**, **CX**, and **SP** after executing the following instructions:

```
PUSH AX
PUSH BX
XCHG AX,CX
POP      CX
PUSH AX
POP BX
```

# Lab Tasks

Task: 3



- Reverse three characters 123, output should look like as follows:  
Hints, use the concept of push and pop.

SCR emulator screen (80x25 chars)

```
Before Reverse
123
After Reverse
321
```

# Lab Tasks

Task: 4



- Take a string from user. Once user hits enters reverse the string given by the user. Must take input from user. Hints: Use the concept of push and pop.

```
?234abcd
dcba432
```



# Books

- Assembly Language Programing and Organization of the IBM PC

Ytha Yu  
Charles Marut



# References

- stack, push, pop
  - [https://www.youtube.com/watch?v=2Gdvz\\_6m5j8&t=236s](https://www.youtube.com/watch?v=2Gdvz_6m5j8&t=236s)
- Defining and Using Procedures, CALL and RET
  - <https://www.youtube.com/watch?v=0dLkvasLSlo>